01 / LAB OVERVIEW

Include Me

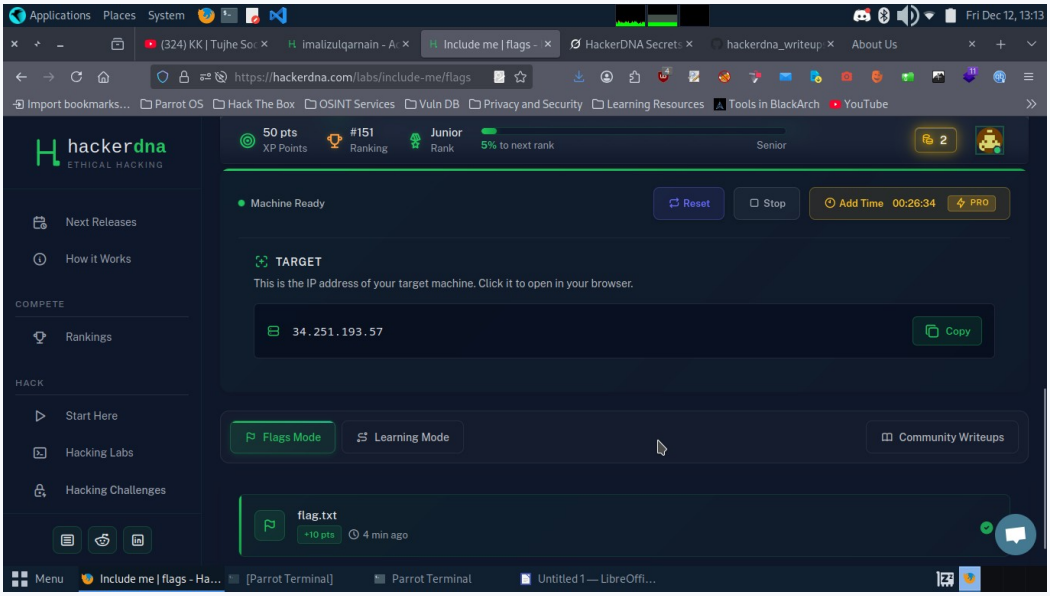
HackerDNA / Local file inclusion and path traversal

English edition / Visual presentation: COMMANDO
A structured adaptation of the supplied walkthrough and its original screenshots.

Objective

Identify an unsafe file-selection parameter in a PHP web application, demonstrate access to unintended local files, and use the same weakness to locate the challenge flag.

Difficulty: Easy **Flags:** 1
Skills: Web reconnaissance, URL parameter testing, LFI and path traversal.



01. Original HackerDNA dashboard for the Include Me lab.

Prepare the instance

Start the lab and copy the target IP into your browser. The recorded session uses **34.251.193.57**; use the address assigned to your own instance. Keep the dashboard available for flag submission.

Follow the evidence

The source screenshots show the application, disclosed system-file contents, and scanner results. The final flag retrieval is described in the source text but is not shown in a separate screenshot.

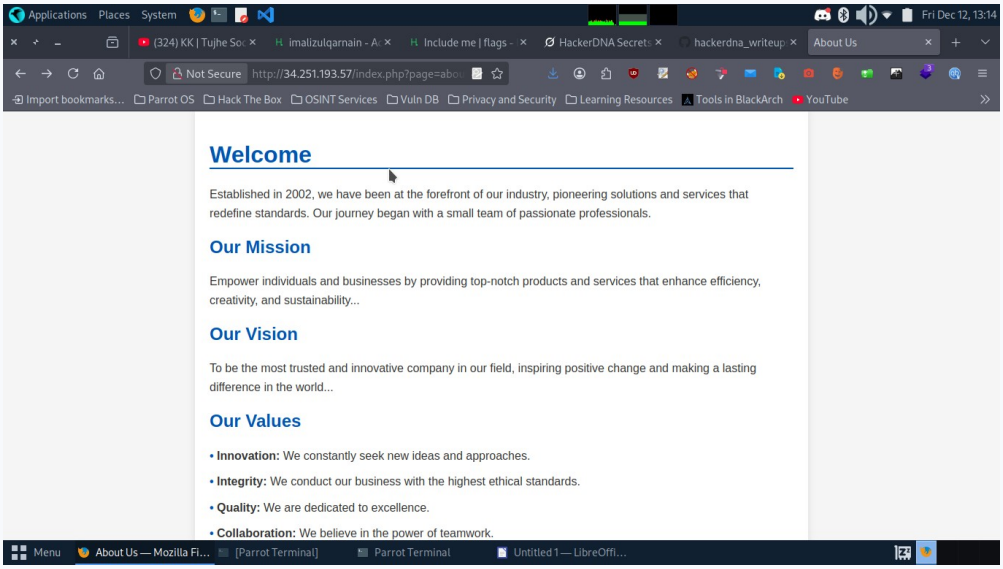
02 / WEB RECONNAISSANCE

Inspect the page parameter

A corporate website with dynamically selected content

The application presents a company website with navigation and informational pages. Following those links reveals a URL parameter that selects the page content.

```
http://<TARGET_IP>/index.php?page=about.html
```



02. Original application page with the page parameter in the URL.

Why this is worth testing

A filename-like parameter may indicate server-side file selection. It is a testing lead, not proof of a vulnerability: a secure implementation could map a small set of page identifiers to fixed files.

Test the boundary

Replace the expected page value with a path outside the intended content directory. The aim is to determine whether the application restricts file selection or uses the submitted path without an adequate boundary check.

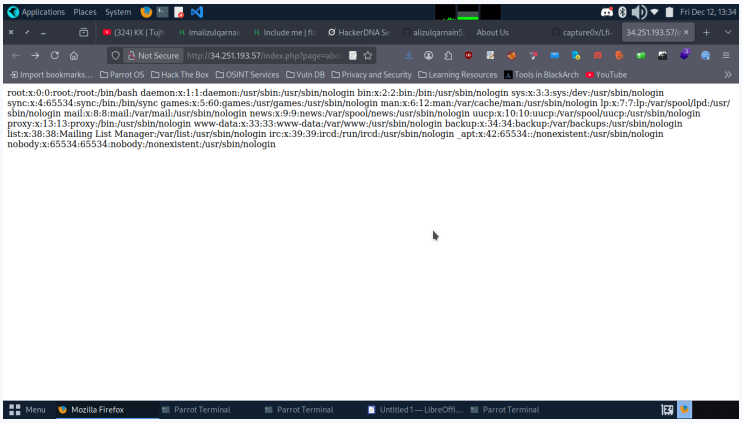
```
http://<TARGET_IP>/index.php?page=../../../etc/passwd
```

This is the manual probe described in the source. The required path depth depends on where the application resolves the filename.

03 / VALIDATION

Confirm local file disclosure

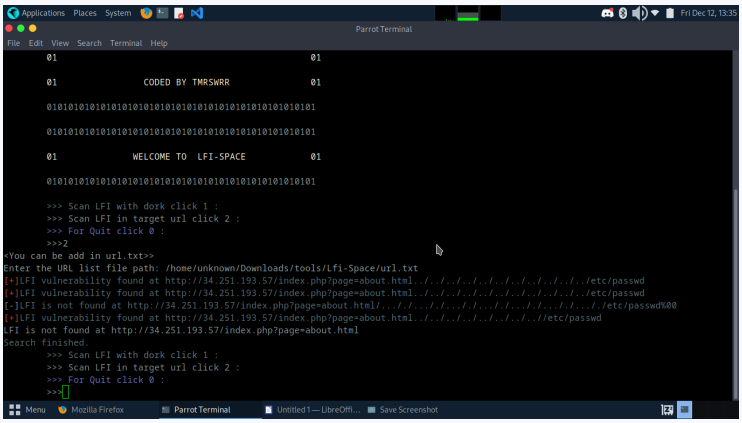
System-file contents provide stronger evidence than errors alone



03. Browser response exposing /etc/passwd-style account records.

The response contains local account records rather than the intended page. This supports unintended local-file access through the page-selection mechanism. It does not demonstrate shell command execution.

Supporting scanner output



04. Original scanner output reporting successful traversal variants.

The scanner tests several path variants against /etc/passwd and reports a mixture of successful and unsuccessful results. The browser disclosure above independently supports the core finding.

Interpret errors carefully

An include warning can reveal how a request was handled, but a permission error is not a successful read. The source mentions null-byte probes; the shown scanner result for the %00 variant is unsuccessful, so it is not part of the confirmed path.

04 / FLAG AND LESSONS

Target the flag file

Reported retrieval path and the underlying trust failure

The path reported in the walkthrough

```
http://<TARGET_IP>/index.php?page=../../../../../flag.txt
```

The original text reports retrieving flag.txt with this path. It contains two parent-directory components (../); the ./ components refer to the current directory. They do not prove that a filter was bypassed.

Use the path supported by your lab instance and inspect the response for the flag. Submit the value in the dashboard. The supplied evidence does not show the final flag contents, so no value is reproduced here.

LFI is not command injection

Submitting a filename such as ls does not inherently execute a shell command. This walkthrough demonstrates file disclosure through a file-selection parameter. Remote code execution would require additional conditions that are not established by the supplied evidence.

Defensive priorities

Map permitted page identifiers to fixed server-side filenames. Reject unexpected identifiers rather than accepting arbitrary paths. If paths must be resolved, canonicalize them and enforce containment within an approved directory.

Keep sensitive files outside readable content locations and apply least-privilege filesystem permissions. PHP configuration restrictions can provide additional protection, but they do not replace a safe file-selection design.

Lessons learned

Inspect navigation parameters, test file boundaries, distinguish warning messages from successful reads, and document the precise evidence. A simple page selector can expose local files when user input is trusted as a filesystem path.

Edition note: English wording refined and all four original screenshots retained. Unsupported escalation claims were removed. Commands and payloads were not re-executed for this edition.